

Mean Normalization

In linear regression, feature scaling helps the algorithm run faster. In recommender systems, a similar technique called **Mean Normalization** is used. It improves computational efficiency, but more importantly, it solves a critical issue regarding **new users** who haven't rated any movies yet.

The Problem: The "Cold Start" for New Users

Let's introduce a new user, **Eve** (User 5), to our dataset. Eve has just signed up and has not rated any movies yet.

- **The Issue:** Because Eve has no ratings ($r(i, j)$ is never 1), the cost function for her parameters does not have any data to fit.
 - **The Result of Regularization:** The regularization term in the cost function tries to keep parameters small. Without data to counteract this, the algorithm will minimize Eve's parameters to zero:
 - $w^{(5)} = [0, 0]$
 - $b^{(5)} = 0$
 - **The Prediction:** If we try to predict Eve's rating for *any* movie:
$$f(x) = w^{(5)} \cdot x^{(i)} + b^{(5)} = 0 \cdot x^{(i)} + 0 = 0$$
The system will predict that Eve hates every single movie (0 stars). This results in terrible recommendations.
-

The Solution: Mean Normalization

Mean normalization forces the "default" prediction for a new user to be the **average rating** of the movie, rather than 0.

Step 1: Calculate Movie Averages

Organize the ratings into a matrix (Y). For each movie (row), calculate the average rating given by users who *have* rated it. Store these in a vector μ .

- *Movie 1 Average:* 2.5
- *Movie 2 Average:* 2.5
- *Movie 3 Average:* 2.0
- ...and so on.

Step 2: Normalize the Data

Subtract the movie average (μ_i) from every existing rating in that row.

$$Y_{norm}^{(i,j)} = Y^{(i,j)} - \mu_i$$

- If User 1 rated Movie 1 a **5**, and the average is **2.5**, the new normalized rating is **2.5**.
- If User 1 rated Movie 4 a **0**, and the average is **2.25**, the new normalized rating is **-2.25**.

Step 3: Learn Parameters

Train the collaborative filtering algorithm using the **new, normalized values** (Y_{norm}) to learn w , b , and x .

Step 4: Make Predictions (Add the Mean Back)

When making a prediction, we calculate the dot product as usual, but we must **add the mean** (μ_i) **back** to get the final star rating.

$$Prediction = (w^{(j)} \cdot x^{(i)} + b^{(j)}) + \mu_i$$

Why This Fixes the Problem

Let's look at **Eve** (User 5) again using this new method.

1. Since she has no data, the regularization still results in $w^{(5)} = [0, 0]$ and $b^{(5)} = 0$.
2. **However**, the prediction formula has changed:
 $Prediction = (0 \cdot x^{(i)} + 0) + \mu_i = \mu_i$
3. **Result:** The system now predicts that Eve will rate a movie equal to the **average rating** that movie received from other users.
 - If a movie is generally liked (Avg 4.5), we predict Eve will give it a 4.5.
 - If a movie is generally disliked (Avg 1.0), we predict Eve will give it a 1.0.

This is a much more reasonable starting point than assuming 0 stars for everything.

Implementation Note: Rows vs. Columns

- **Normalize Rows (Movies):** Helps when you have a **new user**. (Recommended).
- **Normalize Columns (Users):** Helps when you have a **new movie** that no one has rated yet.
 - *Note:* Normalizing columns is less critical because if a movie has 0 ratings, the system shouldn't be recommending it to many people anyway. It is more important to give good recommendations to new users immediately.